UNIVERSIDAD DEL ROSARIO

Escuela de Ingeniería, Ciencia y Tecnología

Ingeniería de Datos **ENTREGABLE 2**

Fase de Diseño Lógico, Físico e Implementación **Diseño de una solución**

de bases de datos y tablero de control para el análisis de rentabilidad de Liwi Flor Eterna

Cliente: Liwi Flor Eterna — Bogotá, Colombia **Integrantes y roles**

Integrante	**Rol en el equipo**	**Responsabilidad principal**
José David	Arquitecto de datos	Modelado conceptual, lógico y físico de la base de datos
Jhonatan	Enlace con el cliente	Levantamiento de requisitos e historias de usuario
Antoine	Facilitador de flujo	Implementación del script, documentación y control de avances

Bogotá D.C., Colombia

Octubre de 2026

Contenido

1 Resumen ejecutivo **2**

2 Correcciones respecto al Entregable 1 **2**
2.1 Primera ronda: incoherencias del Entregable 1 2
2.2 El diagrama de clases, antes y después 3
2.3 Segunda ronda: los datos reales cambiaron el modelo 4
2.4 Requisitos 5

3 Planificación de tareas: historias de usuario **6**
3.1 Lo que sigue pendiente con el cliente 6

4 Modelado de la base de datos **6**
4.1 Modelo de clases 6
4.2 Modelo lógico 9
4.3 Modelo físico 11
4.4 Normalización 12

5 Diccionario de datos **13**

6 Implementación de la base de datos **13**
6.1 Creación de tablas 13
6.2 Carga de datos 14
6.3 Consultas de consulta de datos 15
6.4 Modificación y eliminación 15
6.5 Consultas avanzadas 15
6.6 Vistas, funciones y procedimientos 16
6.7 Triggers 16
6.8 Evidencia de ejecución 16

7 Plan de pruebas **17**

8 Evidencia de pruebas en clientes gráficos **18**
8.1 Caso 1: acceso y bloqueo de una cuenta (MySQL Workbench) 18
8.2 Caso 2: registro de una venta y sus líneas (DBeaver) 20

9 Limitaciones y trabajo siguiente **22**

10 Herramientas utilizadas **23**

11 Documentos que acompañan a este informe **24**

12 Repositorio del proyecto **24**

Referencias **24**

1 Resumen ejecutivo

Liwi Flor Eterna es un negocio bogotano que vende flores de Inírida, vinos, dulces y productos de la despensa amazónica, tanto en un local como fuera de él, y que hasta ahora lleva sus cuentas en dos libros de Excel anotados a mano. El problema que atiende este proyecto es que, con esos libros, el negocio no puede saber con facilidad cuánto vende, cuánto gasta ni qué productos le dejan algo. El Entregable 1 dejó el análisis y un primer modelo conceptual. Este entregable lo lleva hasta una base de datos que corre: la corrige, la baja a un modelo lógico y físico, y la implementa en MySQL con datos del negocio, consultas, vistas, procedimientos y triggers.

Hubo un cambio de fondo a mitad de camino. El modelo con el que se empezó asumía pedidos con anticipo y estados, inventario con alertas y varios puntos de venta. Cuando llegaron los libros de Excel del cliente (el de ingresos y el de gastos) quedó claro que nada de eso existe en el negocio: lo que hay son ventas anotadas a mano, con errores de escritura, y gastos clasificados por concepto. El modelo se rehízo sobre eso, y es el que se entrega.

El resultado es una base de datos de **21 tablas** que se instala desde cero sin errores, con 20 claves foráneas, 18 restricciones de unicidad y 13 restricciones de validación. Se cargaron **500 ventas con 619 líneas y 500 gastos**. De ellos, **96 ventas (122 líneas) y 110 gastos son los reales del cliente**, del 15 de agosto al 15 de septiembre de 2026; el resto es simulado con la misma forma de los libros, para tener volumen de prueba, y el DML trae las sentencias para quitarlo. No se inventaron precios: las 208 líneas sin precio (44 de ellas reales) quedan marcadas como pendientes. Sobre esa base se escribieron 168 consultas por tabla, 8 uniones, 8 subconsultas, 10 vistas, 2 funciones, 12 procedimientos almacenados y 7 triggers. Todo se ejecutó contra el servidor y el documento de evidencia recoge su salida.

El inventario se dejó fuera del alcance por decisión del equipo: ningún libro registra existencias, y modelarlo habría obligado a inventar datos. Los gastos de compra de mercancía sí se enlazan con el producto que se vende, lo que permite una rentabilidad aproximada sin llevar existencias.

La base responde a 65 requisitos funcionales y 27 reglas de negocio, organizados en 21 historias de usuario. Se entregan además el diccionario de datos, el registro de la lógica programada y un guion que ejecuta cada requisito paso a paso contra el servidor. Entre las decisiones de diseño están la separación entre producto y presentación con SKU, los diccionarios de alias que reconocen un mismo producto escrito de varias formas, el cierre financiero como foto del periodo y una bitácora de cambios.

El impacto esperado es que el cliente deje de reconstruir sus resultados a mano: con las vistas consulta ingresos contra gastos por mes y compara lo comprado con lo vendido de cada producto. Quedan por confirmar con él la dirección del local, los costos del local, los precios de las líneas sin valor, el tiempo de inactividad de la sesión y la longitud mínima de la contraseña.

2 Correcciones respecto al Entregable 1

Se hicieron dos rondas de corrección. La primera, antes de implementar, depuró el modelo conceptual. La segunda, al llegar los datos reales, cambió el modelo de fondo.

2.1 Primera ronda: incoherencias del Entregable 1

Antes de avanzar se revisó el Entregable 1 contra lo que se iba a implementar. Aparecieron decisiones de modelado que no sobrevivían al paso a tablas, documentos que se contradecían entre sí y requisitos sin cubrir.

Dónde	Qué estaba mal	Corrección
Diagrama de clases	Usuario y Cliente se modelaron con herencia (tres subclases de usuario, dos de cliente) y existía una clase TableroControl	La jerarquía se unificó con un atributo discriminante (`rol`, `tipoCliente`); el tablero no almacena hechos, así que sale del modelo
Diagrama de clases	Al unificar las jerarquías se perdieron 42 operaciones que vivían en las subclases	Se repartieron entre las clases que poseen los datos que cada una necesita
Diagrama de clases	Faltaba la asociación Usuario–PuntoVenta y sobraba una asociación directa Pedido–Producto	Se corrigieron contra las claves foráneas reales
Modelo conceptual	Faltaban entidades que el sistema necesita (proveedor, usuario, pago, alerta, inventario)	Se incorporaron al modelo lógico; después, con los datos reales, se ajustaron (ver segunda ronda)
Requisitos	Requisitos asignados a un rol (administrador de inventario) que dejó de existir	Pasaron al rol Administrador
Todo el documento	Tres convenciones de nombres de columna distintas entre el diagrama, el script y el diccionario	Se unificó en todo el proyecto: nombres completos, minúscula inicial y mayúscula en cada palabra siguiente
Metodología	Dos pasos del modelado figuraban a cargo de quien no correspondía según la tabla de integrantes	Se corrigió la asignación

2.2 El diagrama de clases, antes y después

La primera figura es el diagrama del Entregable 1 y la segunda el mismo diagrama con la jerarquía de usuarios ya unificada. La tercera es la versión actual, sobre el modelo definitivo (sección 4.1).

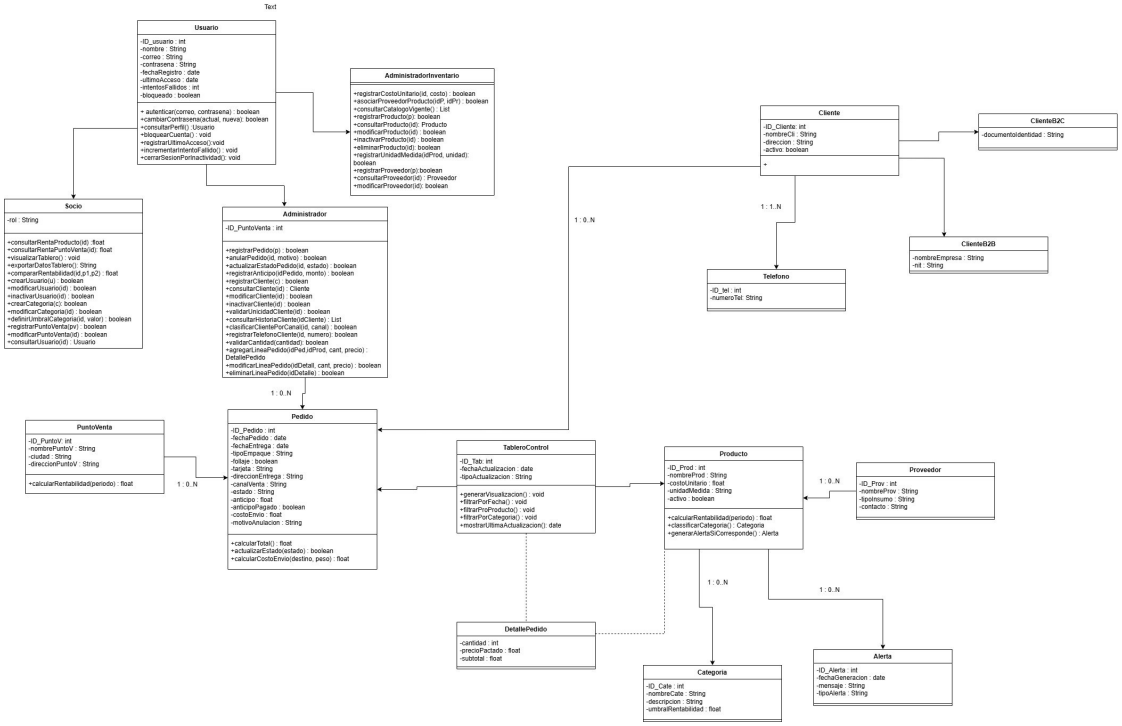


Figura 1: Diagrama de clases del Entregable 1

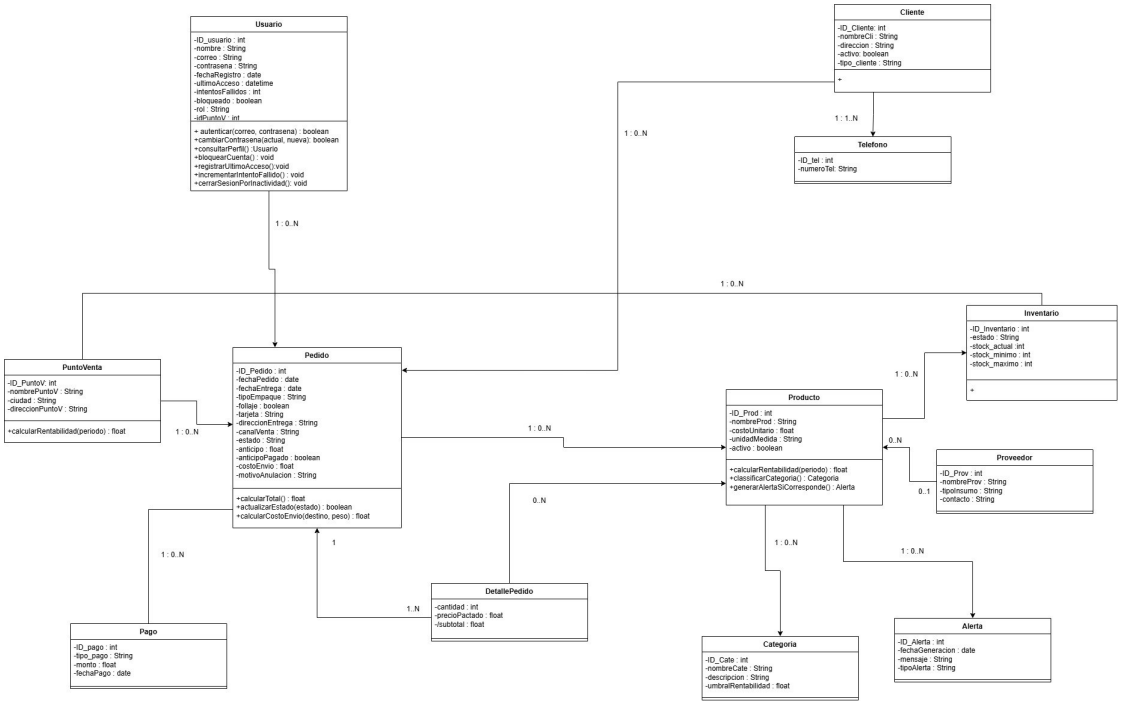


Figura 2: Primera corrección: jerarquía de usuarios unificada

2.3 Segunda ronda: los datos reales cambiaron el modelo

El cliente entregó dos libros de Excel, uno de ingresos y otro de gastos. Al revisarlos se vio que el modelo de pedidos, pagos, inventario y alertas no tenía en qué apoyarse. La tabla resume lo que salió, lo que entró y por qué.

Tema	Antes	Ahora y por qué
Lo que se vende	Pedido, DetallePedido y Pago, con estados y anticipos	`Venta` y `DetalleVenta`. Nadie registra anticipos ni estados: el libro anota la venta ya hecha, con ciudad, método de pago y si fue en el local o fuera
Inventario y alertas	Inventario y Alerta, con stock y umbrales	Fuera del alcance. Ningún libro trae existencias; modelarlas habría obligado a inventar datos
Productos	Una sola tabla Producto	Producto es la familia y `Presentacion` la variante vendible, con su SKU (invierno, verano, grande, pequeño). Las escrituras distintas del mismo producto se absorben en `AliasProducto`
Puntos de venta	Se simularon 50	Existe uno solo, el Local Bogotá. Las 172 ventas que no son del local tienen `idPuntoVenta` vacío
Gastos	Sin módulo	`Gasto`, `ConceptoGasto`, `CategoriaGasto` (Operación, Inventario, Inversión inicial), `TipoDocumento`, `Proveedor`, `MetodoPago` y `Etiqueta` con su tabla puente
Texto mal escrito	Se corregía a mano	Diccionarios de alias para productos y conceptos, y dos funciones que los resuelven
Cierre y auditoría	Fuera del alcance	`CierreFinanciero` (foto congelada de un periodo) y `BitacoraCambio` (alimentada por triggers)

Objeto	Antes	Ahora
Tablas	12	21
Vistas	4 (más 1 de las mejoras)	10
Funciones	1	2
Procedimientos	4 (más 2 de las mejoras)	12
Triggers	3 (más 5 de las mejoras)	7

Tabla 3: Tamaño del modelo antes y después

2.4 Requisitos

Los requisitos se reescribieron sobre el modelo definitivo: **65 requisitos funcionales y 27 reglas de negocio**, organizados en nueve módulos y trazados contra los objetos de la base que los cumplen. Hay un documento detallado y otro con solo la lista. De los 65, 35 los cumple la base por sí sola (restricciones, procedimientos, triggers o vistas); el resto depende de la aplicación, que aún no está construida.

3 Planificación de tareas: historias de usuario

Se escribieron 21 historias de usuario agrupadas en siete épicas. Cada una tiene título, la historia en formato rol–necesidad–beneficio [5], criterios de aceptación, escenarios en formato Gherkin y una estimación en puntos. Cubren los 65 requisitos funcionales y las 27 reglas de negocio. Las historias completas están en el documento *Historias de Usuario*.

Épica	Historias	Requisitos	Puntos
E1. Acceso y usuarios	HU-01 a HU-03	RF-01 a RF-08	13
E2. Configuración y clientes	HU-04 y HU-05	RF-09 a RF-16	8
E3. Catálogo y alias	HU-06 y HU-07	RF-17 a RF-25	10
E4. Ventas	HU-08 a HU-11	RF-26 a RF-37	21
E5. Gastos	HU-12 a HU-16	RF-38 a RF-52	22
E6. Cierre, resultados y tablero	HU-17 a HU-19	RF-53 a RF-61	21
E7. Auditoría y carga	HU-20 y HU-21	RF-62 a RF-65	15
Total	**21 historias**	**65 de 65**	**110**

Tabla 4: Resumen de las historias por épica

La estimación usa la serie de Fibonacci y es una propuesta del equipo de desarrollo, pendiente de validar. Las historias HU-19 y HU-21 y el cierre por inactividad dependen de la aplicación; sus escenarios describen el comportamiento esperado y se probarán cuando exista la interfaz.

3.1 Lo que sigue pendiente con el cliente

Hay datos que el cliente aún no ha confirmado. El modelo está preparado para recibirlos, pero no se inventaron.

- El significado de las etiquetas `LIWI`, `F` y `AKAYUÚ` de los gastos.
- La dirección del local y los costos propios del local (arriendo, luz, internet), de los que depende el resultado parcial del punto de venta.
- El precio de las 208 líneas de venta que el libro no trae.
- El tiempo de inactividad tras el cual se cierra la sesión y la longitud mínima de la contraseña (el equipo propone 8 caracteres).

4 Modelado de la base de datos

4.1 Modelo de clases

El diagrama de clases describe el sistema como objetos con atributos y comportamiento. Es un modelo de diseño, no de datos: una clase tiene métodos y una tabla no. La versión actual representa el modelo definitivo y mantiene los cambios de la primera ronda: cada operación vive en la clase que posee los datos que necesita, y las asociaciones coinciden con las claves foráneas reales.

Tipo de relación	Dónde se usa	Qué significa
Composición (rombo relleno)	Venta–DetalleVenta, Gasto–GastoEtiqueta	La parte no existe sin el todo. Se traduce en `on delete cascade`
Clase de asociación	GastoEtiqueta sobre Gasto–Etiqueta	Relación muchos a muchos
Asociación simple	Las demás	Vínculo sin dependencia de existencia

Tabla 5: Tipos de relación y su correspondencia con la base

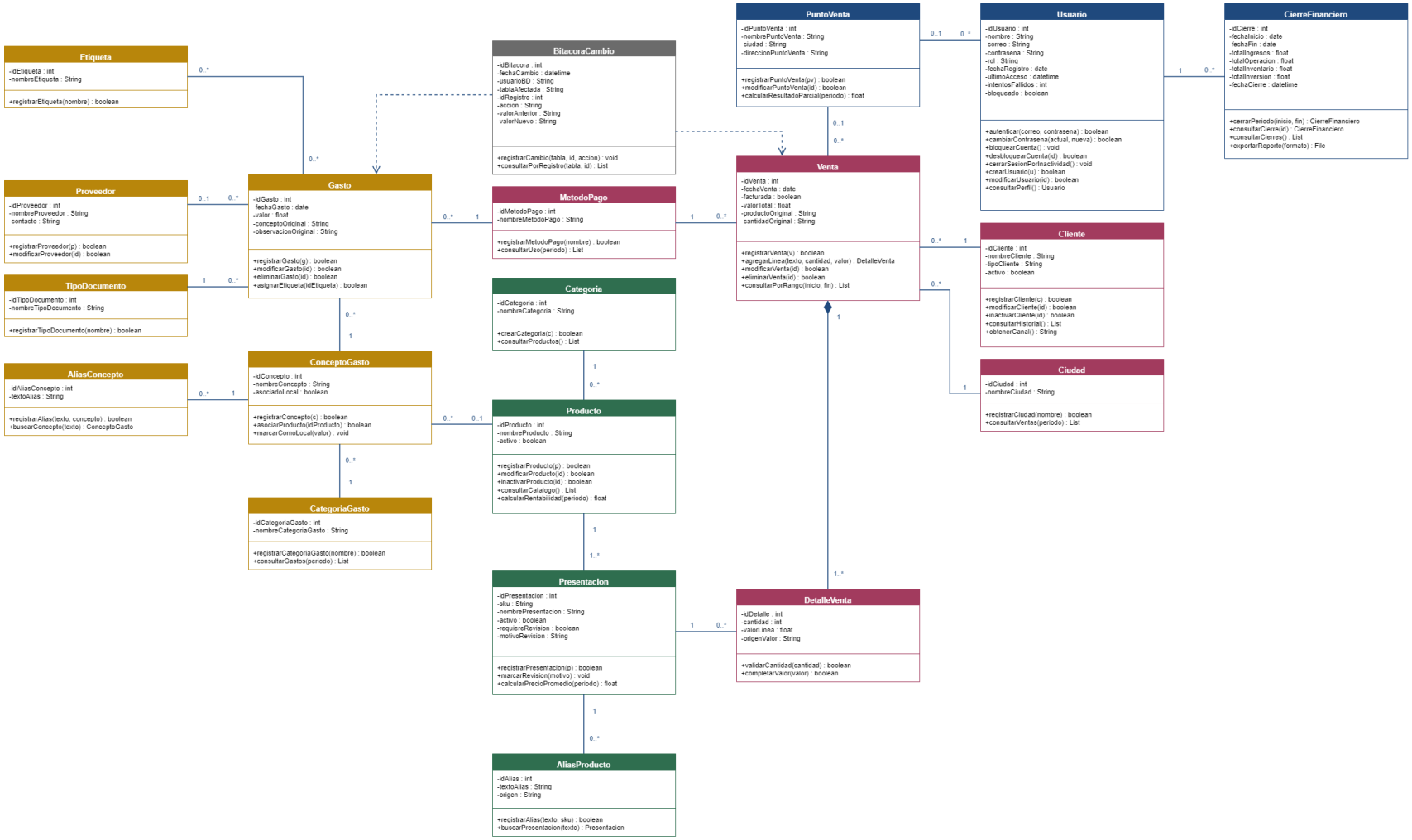


Diagrama de clases · Liwi Flor Eterna

Figura 3: Diagrama de clases actual

4.2 Modelo lógico

El modelo lógico, que baja a tablas el modelo entidad-relación [2], se extrajo de la base real, no se dibujó a mano: así coincide con lo que existe. Muestra 21 tablas con sus claves primarias, foráneas y la regla de borrado de cada una. Los diagramas también se entregan en formato editable de draw.io.

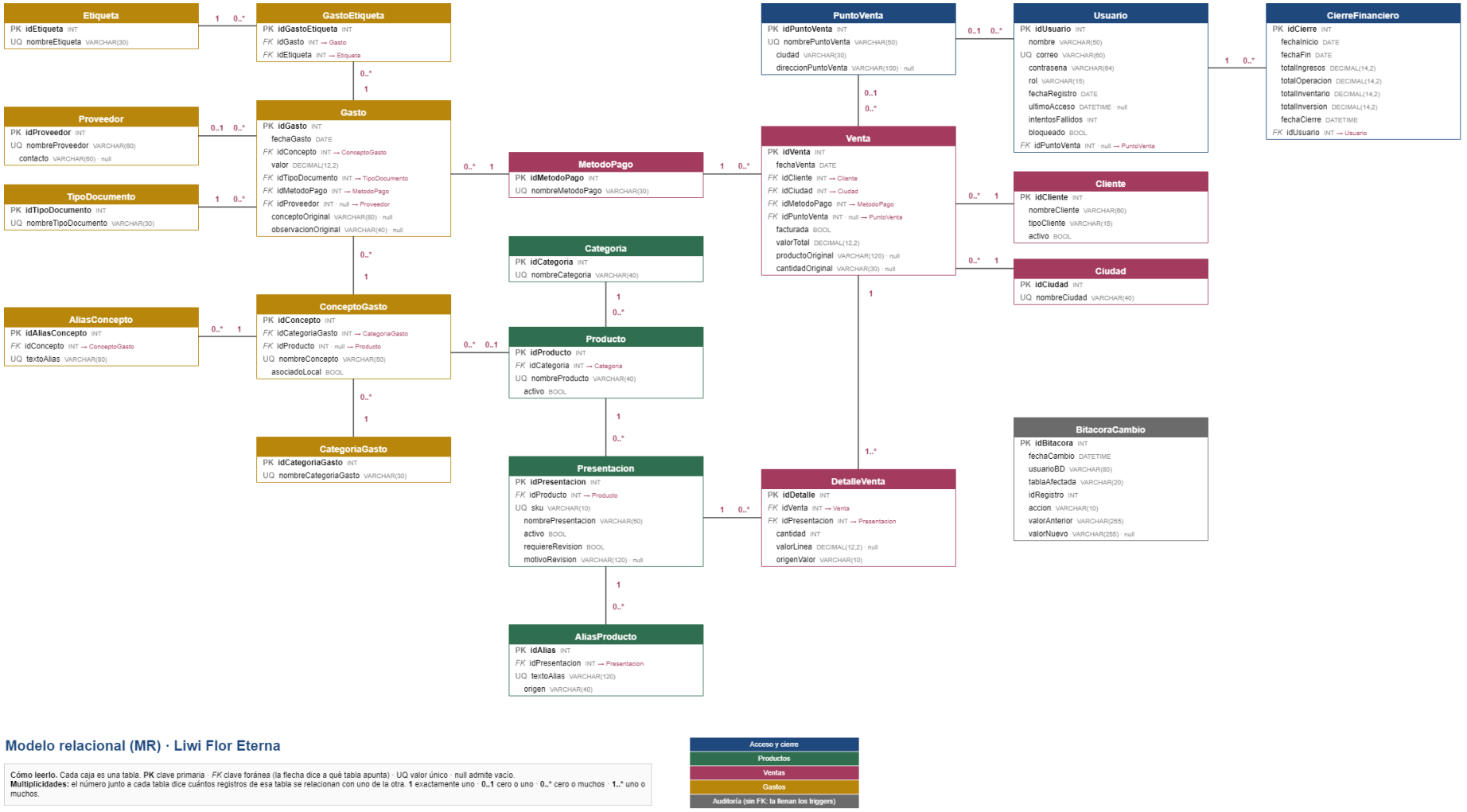


Figura 4: Modelo relacional actual

Las reglas de borrado son deliberadas: de las 20 claves foráneas, 16 son `restrict` (no se puede borrar un cliente que tiene ventas, ni un concepto que tiene gastos), 2 son `cascade` (las líneas de una venta y las etiquetas de un gasto desaparecen con su padre) y 2 son `set null` (si se retira un producto, su concepto de compra queda sin producto; si se retira un proveedor, sus gastos quedan sin proveedor).

4.3 Modelo físico

Aspecto	Decisión	Razón
Motor	InnoDB	Es el único que respeta claves foráneas y transacciones
Juego de caracteres	`utf8mb4`	Los nombres con tildes y eñes se almacenan y comparan bien
Valores monetarios	`decimal(12,2)`	Un float acumula error de redondeo, inaceptable con dinero
Dominios cerrados	Restricciones check	Rol, tipo de cliente, origen del valor y acción de la bitácora tienen valores fijos validados por el motor
Claves primarias	Simples y autoincrementales	Evitan dependencias parciales; las combinaciones naturales quedan protegidas con restricciones de unicidad
Contraseñas	SHA-256 [6]	Se guarda el hash, nunca el texto
Valor de línea	`null` permitido	Una línea sin precio no se inventa: queda `Pendiente`. Un check obliga a que las líneas exactas tengan valor y las pendientes no

Tabla 6: Decisiones del modelo físico

La Figura 5 es el diagrama del modelo físico generado desde el gestor: se obtuvo con la ingeniería inversa de MySQL Workbench sobre la base ya instalada, de modo que muestra exactamente lo que existe en el servidor, con las 21 tablas, sus tipos, sus claves y las 10 vistas. Los nombres aparecen en minúscula porque así los muestra el servidor en que se hizo la captura; las llaves foráneas y sus cardinalidades coinciden con el modelo lógico de la sección anterior.

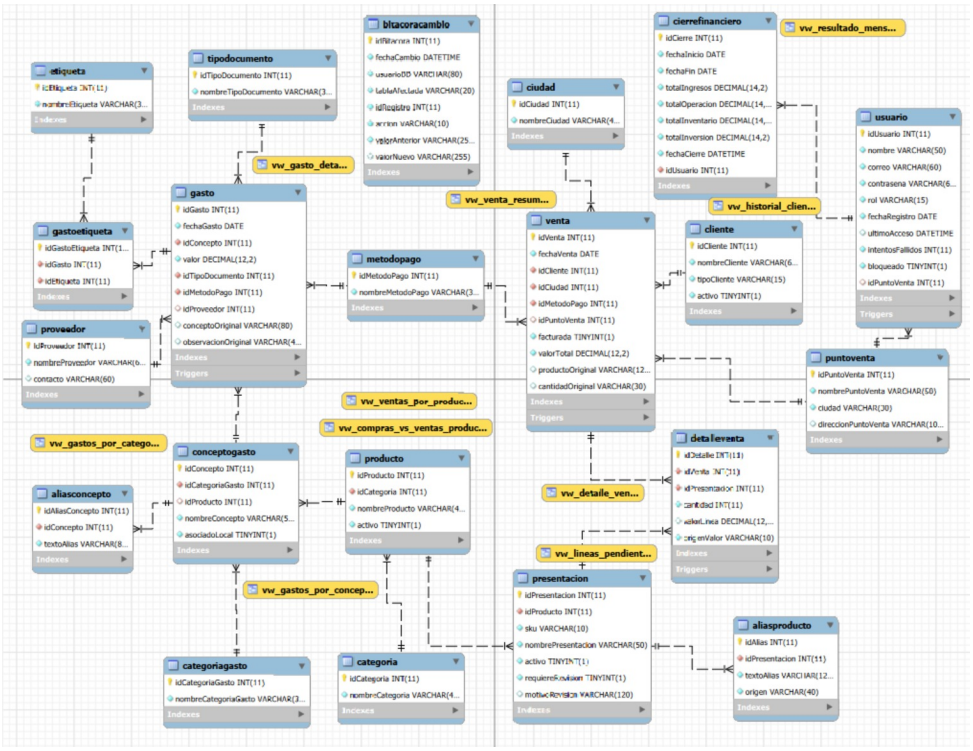


Figura 5: Diagrama del modelo físico generado desde MySQL Workbench (Database → Reverse Engineer)

4.4 Normalización

El modelo cumple la tercera forma normal [1, 3, 4], con algunas desnormalizaciones deliberadas que se justifican abajo.

Primera forma normal. No hay atributos multivaluados. El caso que lo exigía eran las etiquetas de un gasto, que pueden ser varias: se resolvió con la tabla puente `GastoEtiqueta`, donde cada fila es una etiqueta. Del mismo modo, una venta con varios productos se descompone en líneas de `DetalleVenta`.

Segunda forma normal. Todas las claves primarias son simples, así que no puede haber dependencia parcial. En `GastoEtiqueta`, que naturalmente tendría clave compuesta, se usó una clave artificial y la pareja gasto–etiqueta quedó protegida con una restricción de unicidad.

Tercera forma normal. Varias separaciones existen precisamente para evitar dependencias transitivas:

- El producto no repite sus presentaciones: la familia vive en `Producto` y la variante, con su SKU, en `Presentacion`.
- La categoría de un gasto no se guarda en el gasto: sale del concepto, y el concepto apunta a su categoría.
- El canal (B2C o B2B) no se almacena: se deduce del tipo de cliente en la vista `vw_venta_resumen`.

Desnormalización deliberada.

- `Venta.valorTotal` no se calcula como suma de líneas, porque las líneas pendientes no tienen valor. El total es el dato que trae el libro y las líneas son su desglose parcial.
- `productoOriginal`, `cantidadOriginal` y `conceptoOriginal` guardan el texto tal como se escribió en el libro. Se conservan para poder contrastar lo cargado con la fuente.

- `CierreFinanciero` guarda totales que podrían recalcularse. Es a propósito: es una foto congelada del periodo y no debe cambiar aunque después se corrija una venta.
- `PuntoVenta.ciudad` es texto libre y no una clave a `Ciudad`. Con un único local no genera problemas, pero si se abren más conviene unificarlo.

5 Diccionario de datos

El diccionario describe las 21 tablas y sus 101 columnas, con tipo, tamaño, restricciones y descripción. Se generó leyendo `information_schema`, de modo que coincide con lo que existe en la base. Está en el documento *Diccionario de datos*. Como muestra, estas son las dos tablas centrales del flujo de ventas:

Campo	Tipo	Descripción
Venta		
idVenta	int, PK	Identificador de la venta
fechaVenta	date	Fecha de la venta
idCliente	int, FK	Cliente que compró
idCiudad	int, FK	Ciudad de la venta o la entrega
idMetodoPago	int, FK	Forma en que se pagó
idPuntoVenta	int, FK, null	Local donde se hizo; vacío si fue fuera del local
facturada	tinyint(1)	1 si se emitió factura
valorTotal	decimal(12,2)	Valor total; debe ser mayor que cero
productoOriginal	varchar(120), null	Texto del producto tal como estaba en el libro
cantidadOriginal	varchar(30), null	Texto de la cantidad tal como estaba en el libro
DetalleVenta		
idDetalle	int, PK	Identificador de la línea
idVenta	int, FK	Venta a la que pertenece
idPresentacion	int, FK	Presentación vendida
cantidad	int	Unidades; debe ser mayor que cero
valorLinea	decimal(12,2), null	Valor de la línea; vacío si el libro no lo dice
origenValor	varchar(10)	Exacto o Pendiente

6 Implementación de la base de datos

La base se entrega en tres scripts que se ejecutan en orden: `01_DDL.sql` crea las tablas, vistas, funciones y procedimientos; `02_DML.sql` carga los datos y, al final, crea los triggers; `03_Consultas_E2.sql` reúne las consultas de evidencia. Los triggers se crean al final a propósito, para que la carga histórica no dispare la bitácora ni las validaciones.

6.1 Creación de tablas

El DDL crea las 21 tablas con todas sus restricciones. Como ejemplo, estas son las dos tablas centrales:

```sql
create table Venta (
    idVenta          int not null auto_increment,
    fechaVenta       date not null,
    idCliente        int not null,
    idCiudad         int not null,
    idMetodoPago     int not null,
    idPuntoVenta     int null,
    facturada        boolean not null default 0,
    valorTotal       decimal(12,2) not null,
    productoOriginal varchar(120) null,
    cantidadOriginal varchar(30)  null,
    constraint pk_venta primary key (idVenta),
    constraint chk_venta_valor check (valorTotal > 0),
    constraint fk_venta_cliente foreign key (idCliente)
        references Cliente (idCliente) on update cascade,
    ...
) engine = InnoDB;

create table DetalleVenta (
    idDetalle      int not null auto_increment,
    idVenta        int not null,
    idPresentacion int not null,
    cantidad       int not null,
    valorLinea     decimal(12,2) null,
    origenValor    varchar(10) not null default 'Exacto',
    constraint chk_detalle_cantidad check (cantidad > 0),
    constraint chk_detalle_valor check (
        (origenValor = 'Exacto'    and valorLinea is not null and valorLinea > 0)
     or (origenValor = 'Pendiente' and valorLinea is null)),
    constraint fk_detalle_venta foreign key (idVenta)
        references Venta (idVenta) on delete cascade,
    ...
) engine = InnoDB;
```

La base queda con 21 tablas, 10 vistas, 2 funciones, 12 procedimientos, 7 triggers, 20 claves foráneas, 18 restricciones de unicidad y 13 restricciones check. La sección A del documento de evidencia lo consulta directamente en `information_schema`.

6.2 Carga de datos

Los primeros 96 registros de venta y 110 de gasto son los reales de los libros del cliente, con sus errores de escritura absorbidos por los alias y sin inventar valores. El resto hasta completar 500 y 500 es **simulado**: sigue el comportamiento de los libros reales (peso por día de la semana, productos y conceptos más frecuentes, más ventas en diciembre) con una semilla fija, entre el 16 de septiembre de 2026 y el 31 de enero de 2027. Se hizo así para disponer de volumen en las consultas y los cierres. Si se quiere dejar solo lo real, el DML incluye sentencias `delete` que quitan lo simulado.

Tabla	Filas	Tabla	Filas
Venta	500	Presentacion	27
DetalleVenta	619	Producto	13
Gasto	500	Categoria	8
GastoEtiqueta	132	Usuario	6
AliasConcepto	52	TipoDocumento	6
AliasProducto	39	Cliente	7
ConceptoGasto	32	MetodoPago	4
CategoriaGasto	3	Ciudad	3
Etiqueta	3	CierreFinanciero	2
Proveedor	1	PuntoVenta	1
BitacoraCambio	0		

Tabla 8: Registros por tabla tras la carga

Cinco de las 21 tablas superan los 50 registros: las dos de hechos (ventas y gastos con sus detalles) y los alias. Las demás son catálogos pequeños por naturaleza del negocio: hay un solo local, un solo proveedor, tres ciudades y cuatro métodos de pago. Se simuló volumen solo en ventas y gastos; rellenar los catálogos con filas inventadas haría parecer que existe información que el cliente no tiene. `BitacoraCambio` *queda vacía tras la carga a propósito y se llena cuando alguien modifica una venta o un gasto.*

Las comprobaciones del DML confirman que los totales cuadran: las ventas suman 55.522.340 y los gastos 33.517.496, de los cuales lo real es 10.008.440 y 7.723.396. Quedan 208 de las 619 líneas de venta como pendientes. Las ventas se reparten entre 328 en el local y 172 fuera de él.

6.3 Consultas de consulta de datos

Se escribieron ocho consultas por cada una de las 21 tablas, 168 en total, con esta variedad: consulta básica, filtro, `like`, agrupación, `having`, cálculo, orden con rango y `distinct` con `limit`. Todas se ejecutaron: ninguna dio error ni resultado vacío. Un ejemplo, las ventas por mes:

```
select date_format(fechaVenta, '%Y-%m') as mes,
       count(*) as ventas, sum(valorTotal) as total
from Venta group by mes order by mes;
```

6.4 Modificación y eliminación

Se ejecutaron de verdad cinco `update` y un `delete`. Ninguno toca valores monetarios, para que los totales de ventas y gastos sigan cuadrando. Los que afectan ventas o gastos dejan su rastro en `BitacoraCambio` mediante los triggers.

Tipo	Qué hace	Filas
update	Corregir el método de pago de ventas de cliente final registradas como efectivo	1
update	Marcar como facturadas las ventas del 2026-08-20	7
update	Pasar a efectivo los gastos de transporte de la primera semana	7
update	Dejar constancia en el origen de un alias de producto	1
update	Registrar el último acceso de una administradora	1
delete	Eliminar un gasto de prueba creado justo antes	1

Tabla 9: Modificaciones ejecutadas

6.5 Consultas avanzadas

Ocho uniones y ocho subconsultas. Las uniones llegan a juntar hasta cuatro tablas (ventas con cliente, ciudad y método de pago; gastos con concepto, categoría y tipo de documento) e incluyen `left join` y la relación muchos a muchos de las etiquetas. Las subconsultas usan `in`, `exists`, comparación con el promedio, subconsulta correlacionada y subconsulta en el `from`. Un ejemplo, los gastos que superan el promedio de su propia categoría:

```
select g.idGasto, co.nombreConcepto, g.valor
from Gasto g join ConceptoGasto co on co.idConcepto = g.idConcepto
where g.valor > (select avg(g2.valor) from Gasto g2
                 join ConceptoGasto c2 on c2.idConcepto = g2.idConcepto
                 where c2.idCategoriaGasto = co.idCategoriaGasto)
order by g.valor desc limit 10;
```

6.6 Vistas, funciones y procedimientos

Las diez vistas, las dos funciones y los doce procedimientos están descritos uno a uno, con su código y su comprobación, en el documento *Registro de lógica programada*. En resumen:

- **Funciones.** `fn_buscar_presentacion` y `fn_buscar_concepto` traducen un texto escrito a mano a su presentación o su concepto.
- **Procedimientos de escritura.** Registran ventas, líneas, gastos y alias validando cada dato; `sp_completar_linea_venta` pone el precio a una línea pendiente.
- **Acceso y cierre.** `sp_autenticar`, `sp_cambiar_contrasena`, `sp_desbloquear_usuario` y `sp_cerrar_periodo`, estos dos últimos reservados a socios.
- **Consulta.** `sp_rentabilidad_producto` y `sp_rentabilidad_punto_venta`.
- **Vistas.** Ventas (resumen, detalle, por producto, líneas pendientes, historial de cliente), gastos (detalle, por categoría, por concepto) y resultados (compras contra ventas y resultado mensual).

6.7 Triggers

Siete triggers hacen cumplir reglas sin depender de la aplicación.

Trigger	Cuándo	Qué hace
`trg_usuario_bloqueo_automatico`	Antes de modificar Usuario	Bloquea la cuenta al tercer intento fallido
`trg_detalle_valor_coherente` (y `_upd`)	Al insertar o modificar una línea	Impide que una línea valga más que su venta
`trg_venta_bitacora_modifica`	Después de modificar Venta	Guarda cómo era y cómo quedó
`trg_venta_bitacora_elimina`	Después de eliminar Venta	Guarda cómo era
`trg_gasto_bitacora_modifica`	Después de modificar Gasto	Guarda cómo era y cómo quedó
`trg_gasto_bitacora_elimina`	Después de eliminar Gasto	Guarda cómo era

Tabla 10: Triggers de la base

6.8 Evidencia de ejecución

El documento *Evidencia de ejecución* se genera con un programa que reconstruye la base desde cero con los dos primeros scripts, ejecuta el tercero bloque por bloque y arma el informe con la salida literal del servidor. Así no depende de capturas hechas a mano.

Comprobación	Resultado
Bloques ejecutados	246
Consultas con error inesperado	0
Consultas con resultado vacío	0
Operaciones inválidas rechazadas por la base	12 de 12
Instalación de los scripts desde cero	sin errores

Tabla 11: Resumen de la evidencia

Los rechazos esperados incluyen un valor de línea mayor que su venta, una contraseña incorrecta, un administrador que intenta desbloquear o cerrar, un periodo que se cruza con otro, un tipo de cliente inválido, un socio con punto de venta, un gasto negativo, una venta en cero, una acción de bitácora inválida y el borrado de un cliente con ventas. En cada caso el mensaje de error que muestra la evidencia es el resultado correcto.

7 Plan de pruebas

El plan completo está en el repositorio (`tests/plan_de_pruebas.md`) y se resume aquí. El objetivo es comprobar que la base cumple los requisitos que le corresponden y que rechaza los datos que no debe aceptar. Se prueban las reglas que viven en la base de datos (restricciones, funciones, procedimientos, triggers y vistas); lo que depende de la aplicación se probará cuando exista la interfaz, en el Entregable 3.

Hay tres tipos de prueba: positivas (la operación válida funciona), negativas (la base rechaza el dato inválido con el mensaje correcto) y de regla automática (un trigger o una restricción actúa sin que nadie lo invoque). Cada prueba se puede repetir sin dejar cambios, porque cada bloque termina en `rollback`. Una prueba queda aprobada cuando el resultado obtenido coincide con el esperado; en las negativas, cuando el mensaje de error es el previsto. Se ejecutaron el 21 de octubre de 2026 sobre MariaDB 10.11, con la base instalada desde los dos primeros scripts.

ID	RF	Prueba	Resultado esperado	Estado
PP-01	RF-01	Ingresar con la contraseña correcta	Devuelve id y rol; intentos en 0	Aprobada
PP-02	RF-01	Contraseña incorrecta	Rechaza sin decir si falló el correo o la clave	Aprobada
PP-03	RF-01	Bloqueo al tercer intento	Cuenta bloqueada por trigger; no entra	Aprobada
PP-04	RF-02	Solo un socio desbloquea	Rechazo	Aprobada
PP-05	RF-04	Contraseña nueva demasiado corta	Rechazo	Aprobada
PP-06	RF-05	Un administrador no cierra periodos	Rechazo	Aprobada
PP-07	RF-06	Socio con punto de venta	Rechazo por restricción	Aprobada
PP-08	RF-11	Tipo de cliente inválido	Rechazo por restricción	Aprobada
PP-09	RF-14	Vender a un cliente inactivo	Rechazo	Aprobada
PP-10	RF-19	SKU repetido	Rechazo por unique	Aprobada
PP-11	RF-23	Alias hacia un SKU que no existe	Rechazo	Aprobada
PP-12	RF-24	Reconocer un producto con mayúsculas y espacios	Devuelve la presentación de «Flores de invierno»	Aprobada
PP-13	RF-26	Ciudad inexistente	Rechazo	Aprobada
PP-14	RF-26	Total de venta negativo	Rechazo por restricción	Aprobada
PP-15	RF-29	Producto no reconocido en una línea	Rechazo	Aprobada
PP-16	RF-29	Línea mayor que la venta	Rechazo	Aprobada
PP-17	RF-29	Línea sin precio queda pendiente	valorLinea vacío y origenValor = Pendiente	Aprobada
PP-18	RF-30	Completar una línea pendiente	Pasa a Exacto con ese valor	Aprobada
PP-19	RF-32	Eliminar una venta	Se borran sus líneas en cascada y queda rastro	Aprobada
PP-20	RF-33	Rastro al modificar un gasto	Una fila en la bitácora con el valor anterior	Aprobada
PP-21	RF-44	Concepto no reconocido	Rechazo	Aprobada

ID	RF	Prueba	Resultado esperado	Estado
PP-22	RF-45	Categoría automática del gasto	Toma su categoría del concepto, sin digitarla	Aprobada
PP-23	RF-46	Etiqueta repetida en el mismo gasto	Rechazo por unique	Aprobada
PP-24	RF-49	Eliminar un gasto con etiqueta	Se borran sus etiquetas en cascada	Aprobada
PP-25	RF-53	Cierre que se cruza con otro	Rechazo	Aprobada
PP-26	RF-53	Fechas invertidas	Rechazo	Aprobada
PP-27	RF-53	La foto del cierre no cambia	El total del cierre no cambia	Aprobada
PP-28	RF-55	Resultado mensual	Una fila por mes con datos	Aprobada
PP-29	RF-65	Informar lo que no se reconoce en una carga	Aparece en el reporte, no se inserta	Aprobada

Tabla 13: Casos de prueba y su resultado

Los resultados literales que devolvió el servidor, y el guion que recorre los 65 requisitos, están en la carpeta `tests/casos_de_prueba` del repositorio y en el documento de evidencia.

8 Evidencia de pruebas en clientes gráficos

Además del documento de evidencia generado por programa, se ejecutaron dos casos de prueba a mano, en clientes gráficos, y se guardaron las capturas. El primero, hecho en MySQL Workbench, recorre el acceso y el bloqueo de una cuenta (RF-01 y RF-02). El segundo, hecho en DBeaver, recorre el registro de una venta con sus líneas, las líneas pendientes y los rechazos (RF-26, RF-29 y RF-34).

8.1 Caso 1: acceso y bloqueo de una cuenta (MySQL Workbench)

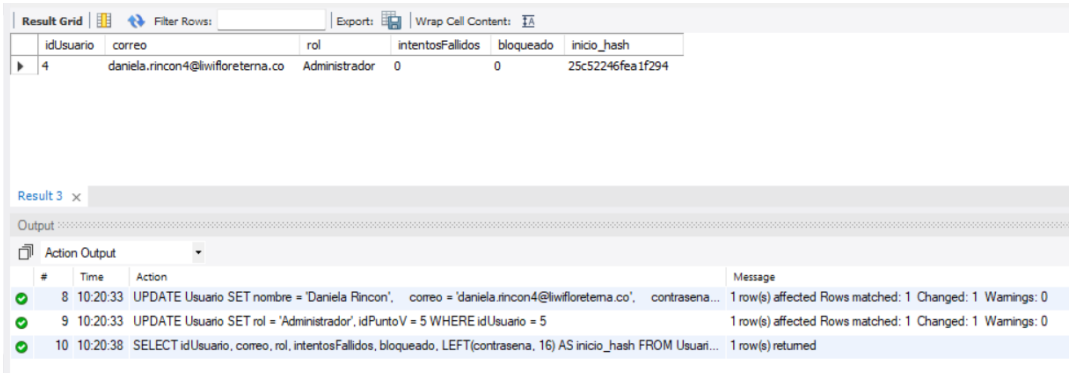


Figura 6: Estado inicial del usuario 4: sin intentos fallidos ni bloqueo, con la contraseña guardada como hash.

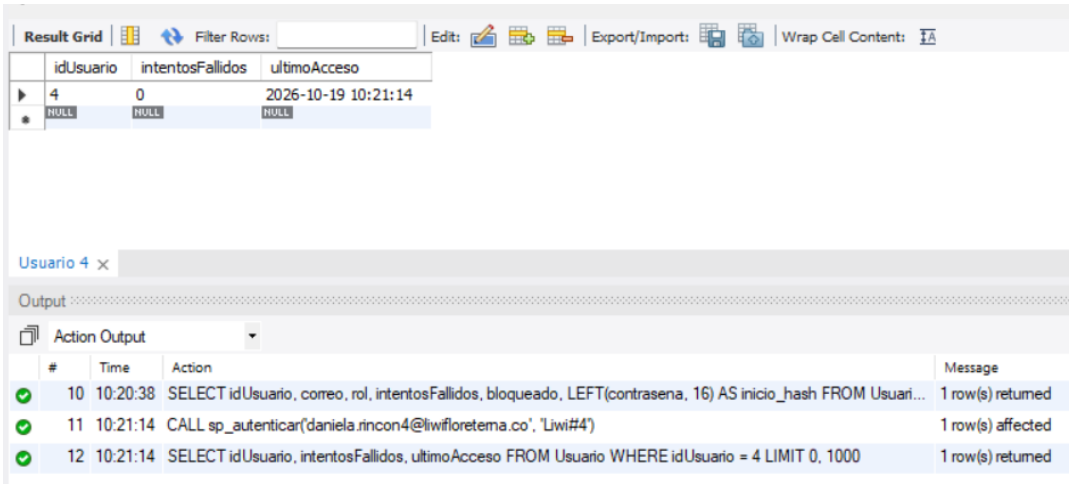


Figura 7: Inicio de sesión correcto con `sp_autenticar`: queda registrado el último acceso.

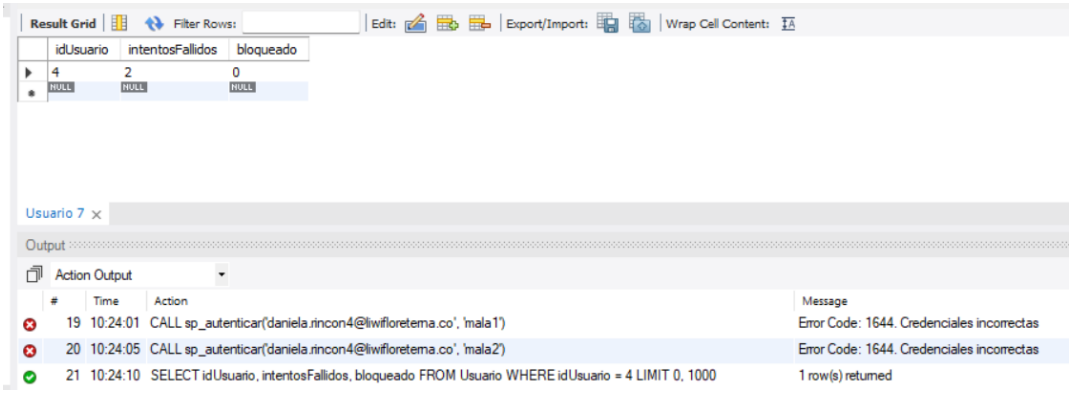


Figura 8: Intentos con contraseña incorrecta: la base rechaza y suma los intentos fallidos.

Figura 9: Correo que no existe: la base rechaza igual que con una contraseña incorrecta.

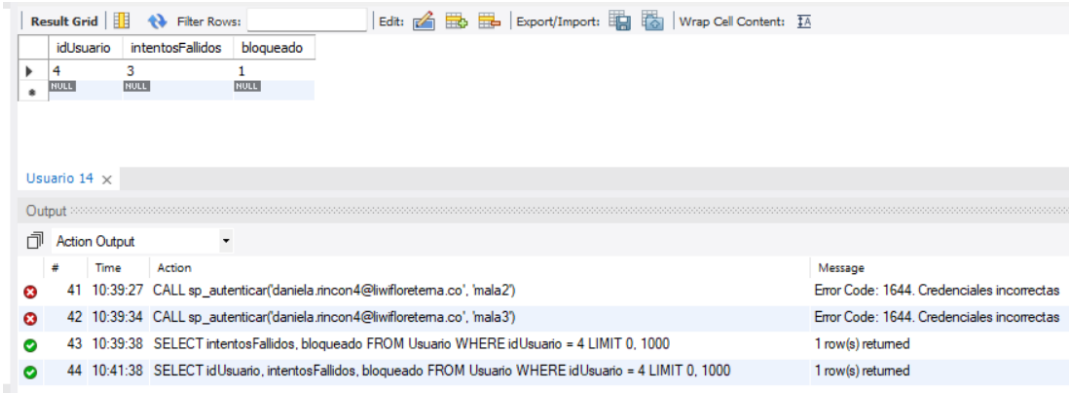


Figura 10: Tercer intento fallido: el trigger bloquea la cuenta (`intentosFallidos = 3`, `bloqueado = 1`).

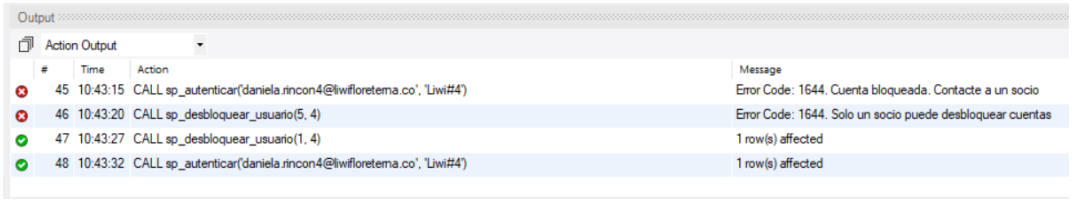


Figura 11: Cuenta bloqueada: no entra ni con la clave correcta; un administrador no puede desbloquearla y un socio sí.

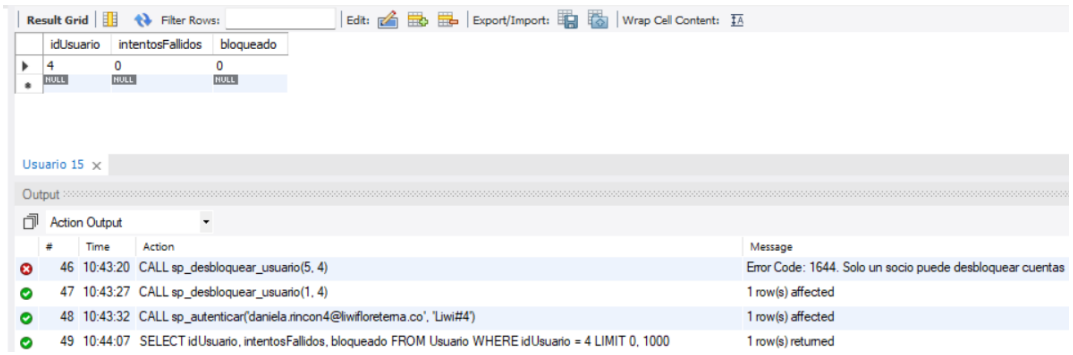


Figura 12: Estado final: intentos en cero y cuenta desbloqueada.

8.2 Caso 2: registro de una venta y sus líneas (DBeaver)

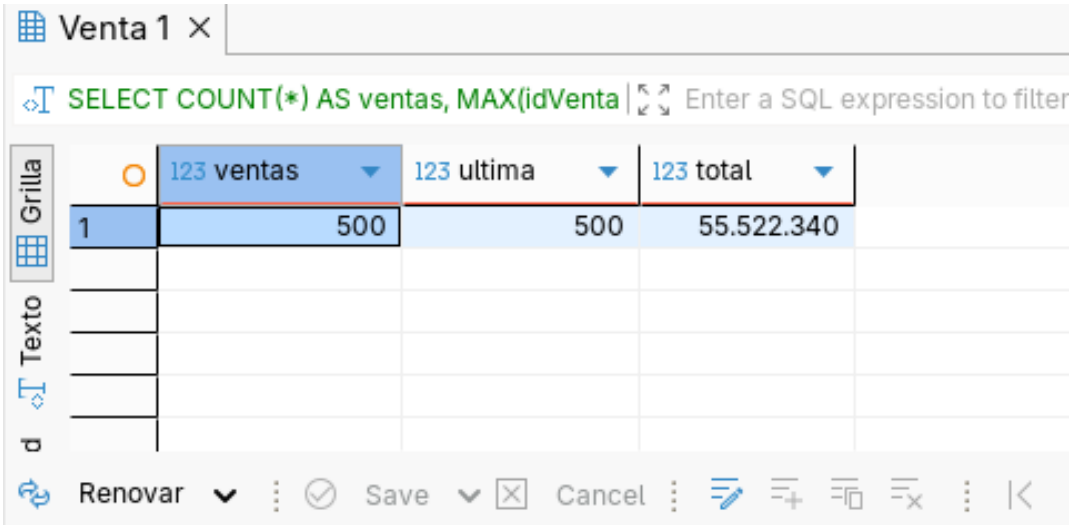


Figura 13: Estado inicial: 500 ventas y un total de 55.522.340.



Figura 14: Alta de una línea con `sp_agregar_linea_venta`: el procedimiento devuelve el identificador de la línea.

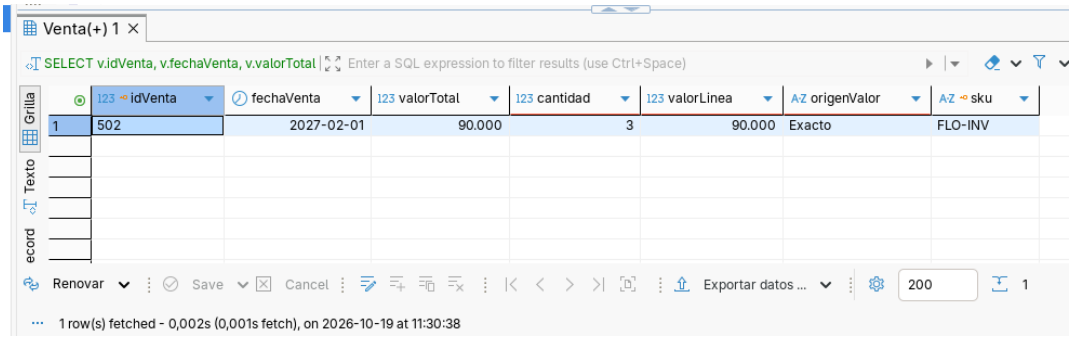


Figura 15: Venta 502 registrada con su línea de valor exacto (FLO-INV, 3 unidades, 90.000).

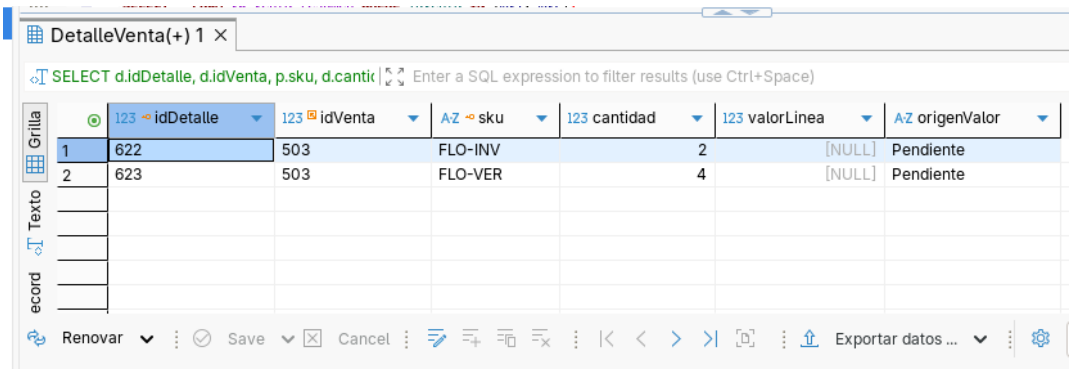


Figura 16: Venta 503 con dos líneas sin precio: quedan como `Pendiente` y sin valor, nunca con un precio inventado.

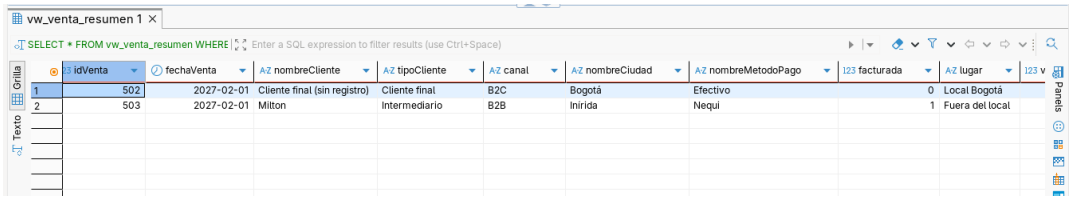


Figura 17: Consulta de `vw_venta_resumen` con las dos ventas nuevas, una en el local y otra fuera de él.

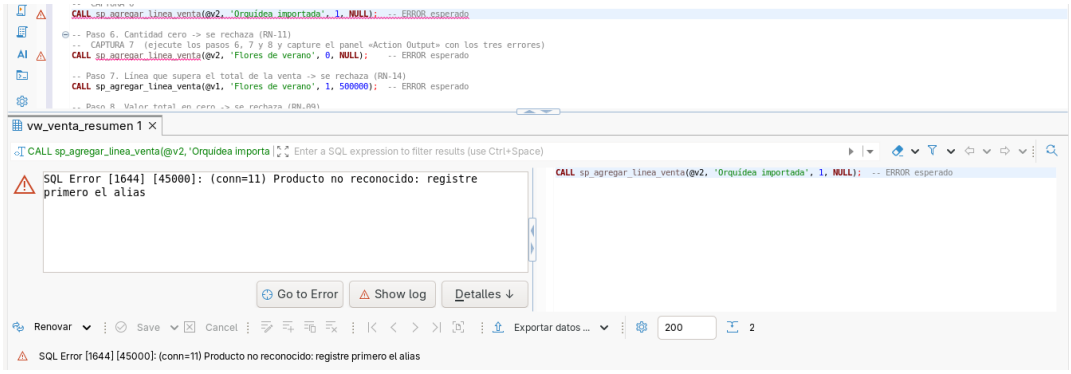


Figura 18: Producto no reconocido: el procedimiento pide registrar primero el alias.

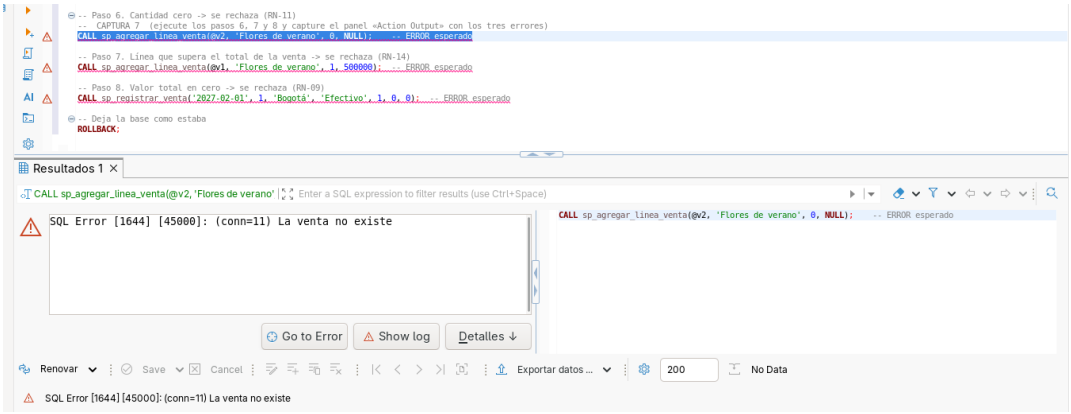


Figura 19: Segunda prueba de rechazo ejecutada en DBeaver, con el mensaje que devolvió la base.

9 Limitaciones y trabajo siguiente

La rentabilidad es aproximada

Los gastos de compra se enlazan con un producto, pero el libro no dice qué parte de una compra se vendió. Además, 208 líneas de venta no tienen precio y no suman al valor vendido exacto. Por eso las cifras de rentabilidad se rotulan como aproximadas.

El resultado del local es parcial

Los costos del local y su dirección están pendientes del cliente. Además, el procedimiento toma como gastos del local todos los marcados como tales, sin distinguir punto de venta: es correcto con un solo local, pero habría que ligar los gastos al punto de venta si se abre otro.

No hay inventario

Es una decisión de alcance. Si el cliente empieza a llevar existencias, el modelo puede ampliarse con una tabla de movimientos sobre `Presentacion`.

La bitácora solo cubre ventas y gastos
Son las dos tablas con dinero. Los cambios en usuarios, clientes o catálogos no dejan rastro.

No hay índices de rendimiento deliberados
Los índices existentes vienen de las claves y las restricciones de unicidad. Con 500 ventas y 500 gastos no se nota, pero conviene definirlos antes de crecer.

La aplicación no está construida
El tablero, la exportación del cierre, la carga desde Excel y el cierre de sesión por inactividad dependen de ella. La base ya expone las vistas que usarían.

Una parte de los datos es simulada
Solo 96 ventas y 110 gastos son reales (15 de agosto a 15 de septiembre de 2026). Todo lo posterior, hasta enero de 2027, es simulado: por eso el resultado mensual llega hasta ese mes y el segundo cierre financiero se calcula casi por completo sobre datos simulados. Las cifras de rentabilidad y de tablero son demostrativas hasta que se cargue más información real.

10 Herramientas utilizadas

Herramienta	Versión	Para qué
MariaDB Server [8]	10.11	Servidor donde se ejecutó y verificó todo el código
MySQL Workbench	—	Diseño y ejecución del script, y diagrama del modelo físico por ingeniería inversa
information_schema	Incluido	Verificación automática del modelo y del diccionario contra la base real
draw.io	—	Diagrama de clases y modelo relacional
LaTeX con pdfLaTeX	TeX Live	Este informe y los documentos de requisitos
Git y GitHub	—	Control de versiones del proyecto

Tabla 14: Herramientas utilizadas en esta fase

Se trabajó sobre MariaDB porque es el servidor disponible en el entorno de desarrollo, y el código se escribió dentro del subconjunto común con MySQL 8 [7]: restricciones check, signal, manejadores de error y triggers se comportan igual en los dos. No se usaron extensiones propias de MariaDB.

11 Documentos que acompañan a este informe

Archivo	Contenido
01_DDL.sql	Tablas, vistas, funciones y procedimientos
02_DML.sql	Carga de datos y creación de los triggers
03_Consultas_E2.sql	Las consultas de evidencia
04_Ejecucion_65_RF.sql y 04_Ejecucion_65_RF_simple.sql	Ejecución explicada de los 65 requisitos funcionales, con y sin procedimiento auxiliar
05_Invocar_RF.sql	Invocación de cada requisito con `call sp_rf_NN();`
Salida_esperada_65_RF	Resultado esperado de los 65 requisitos (txt, tex y pdf)
Plan de pruebas (`tests/plan_de_pruebas.md`)	29 casos de prueba con resultado esperado
Capturas de pruebas	14 capturas en Workbench y DBeaver (carpeta `capturas`)
Diccionario de datos	Las 101 columnas de las 21 tablas
Historias de usuario	Las 21 historias con criterios, escenarios y estimación
Registro de lógica programada	Funciones, procedimientos, triggers y vistas, con su código
Evidencia de ejecución	Salida literal del servidor para los 246 bloques
Requisitos funcionales (detallado y lista)	65 requisitos funcionales y 27 reglas de negocio
Diagramas	Diagrama de clases, modelo relacional y diagrama físico de Workbench, editables en draw.io e imagen

Tabla 15: Archivos entregados

12 Repositorio del proyecto

El código, los diagramas y la documentación están versionados en el repositorio del equipo:

github.com/Davidx34/Proyecto_ingenieria_de_datos

Referencias

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] P. P. Chen, "The entity-relationship model—toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[5] M. Cohn, *User Stories Applied: For Agile Software Development*. Addison-Wesley, 2004.

[6] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[7] Oracle Corporation, *MySQL 8.0 Reference Manual*. Disponible en: https://dev.mysql.com/doc/refman/8.0/en/

[8] MariaDB Foundation, *MariaDB Knowledge Base*. Disponible en: https://mariadb.com/kb/en/